

Django Migration Workflow — bSecure

Table of Contents

1. [Overview](#)
 2. [Initial Setup](#)
 3. [Django Migration Workflow](#)
 4. [Zero-Downtime Migration Patterns](#)
 5. [Data Migrations](#)
 6. [PostGIS-Specific Migrations](#)
 7. [Squashing Migrations](#)
 8. [Migration Naming Conventions](#)
 9. [Multi-Node Deployment](#)
 10. [Rollback Strategy](#)
 11. [Migration Testing](#)
-

1. Overview

Django migrations are the **single source of truth** for all schema changes in bSecure. Every change to a model — adding a field, changing a field type, adding a constraint, creating an index — must go through a Django migration. Direct ALTER TABLE statements run in psql without a corresponding migration will cause migrate --check to fail in CI and will break new environment setups.

Key Principles

Principle	Rule
One source of truth	Schema lives in migration files, not in ad-hoc SQL scripts
Zero-downtime by default	Every migration must be deployable without stopping the application

Principle	Rule
Forward-only in production	Rollbacks are done by writing forward migrations; never migrate app 0001 in prod unless it's a genuine emergency
PostGIS is a first-class citizen	Geography columns, GiST indexes, and spatial functions require PostGIS extensions installed before the first migration runs
Squash when necessary	Squash per-app when an app exceeds ~50 migrations to keep migrate fast

2. Initial Setup

INSTALLED_APPS

`django.contrib.gis` must appear **before** any app that uses `PointField`, `PolygonField`, or other geometry fields. The GeoAdmin and GeoDjango form widgets depend on it.

```
# settings/base.py
INSTALLED_APPS = [
    # Django core
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # GeoDjango – must be before any app using geography fields
    "django.contrib.gis",

    # Third-party
    "rest_framework",
    "corsheaders",
    "django_filters",
    "celery",
```

```

# bSecure apps
"apps.users",
"apps.guards",
"apps.bookings",
"apps.tracking",
"apps.payments",
"apps.reviews",
"apps.safety",
"apps.notifications",
]

```

Database Configuration

```

# settings/base.py
DATABASES = {
    "default": {
        # PostGIS backend – required for
        # PointField(geography=True) and
        # all ST_* functions. This replaces the standard psycopg2
        # backend.
        "ENGINE": "django.contrib.gis.db.backends.postgis",
        "NAME": env("DB_NAME"),
        "USER": env("DB_USER"),
        "PASSWORD": env("DB_PASSWORD"),
        "HOST": env("DB_HOST"),
        "PORT": env("DB_PORT", default="5432"),
        "OPTIONS": {
            "connect_timeout": 10,
        },
        "CONN_MAX_AGE": 60, # persistent connections
    }
}

```

PostgreSQL Extension Setup

Three extensions are required before any table is created:

Extension	Purpose
postgis	Geography/geometry types, ST_* functions, GiST spatial indexes
pg_trgm	Trigram similarity for fuzzy text search (% operator, similarity())
btree_gist	Allows GiST indexes on scalar types (needed for EXCLUDE range constraints)

Install them manually on a fresh database:

```
-- Run as a PostgreSQL superuser (or RDS master user):  
CREATE EXTENSION IF NOT EXISTS postgis;  
CREATE EXTENSION IF NOT EXISTS pg_trgm;  
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

Custom Initial Migration

For automated environment setup (CI, new developer machines, Docker), encode the extension creation in a migration so that `manage.py migrate` sets up a fully functional database from scratch.

```
# apps/users/migrations/0001_initial_extensions.py  
from django.db import migrations  
  
class Migration(migrations.Migration):  
    """  
    Create required PostgreSQL extensions before any tables are  
    created.  
  
    This migration must be the first migration in the dependency  
    chain.  
    It runs with superuser credentials – on RDS, use the master  
    user for  
    the initial setup or pre-create extensions out of band.  
    """  
  
    initial = True  
    dependencies = []  
  
    operations = [  
        migrations.RunSQL(  
            sql="""  
                CREATE EXTENSION IF NOT EXISTS postgis;  
                CREATE EXTENSION IF NOT EXISTS pg_trgm;  
                CREATE EXTENSION IF NOT EXISTS btree_gist;  
            """,  
            reverse_sql="""  
                -- Do not drop extensions on rollback – other apps  
                may depend on them.  
                -- If you need a clean slate, drop the database  
                and recreate it.  
                SELECT 1;  
            """,  
        ),  
    ]
```

3. Django Migration Workflow

Core Commands

Generate migrations from model changes (always review before committing):

```
python manage.py makemigrations
```

Generate migrations for a specific app only:

```
python manage.py makemigrations guards
```

Apply all pending migrations:

```
python manage.py migrate
```

Apply migrations for a specific app only:

```
python manage.py migrate guards
```

Show all migrations and their applied/unapplied status:

```
python manage.py showmigrations
```

Show migrations for specific apps:

```
python manage.py showmigrations guards bookings payments
```

Sample showmigrations output:

```
guards
```

```
[X] 0001_initial_extensions
```

```
[X] 0002_initial
```

```
[X] 0003_add_search_vector
```

```
[X] 0004_add_tier_field
```

```
[ ] 0005_add_current_location  ← not yet applied
```

```
bookings
```

```
[X] 0001_initial
```

```
[X] 0002_add_payment_status
```

Fake Migrations

Use fake migrations when the database schema already matches the migration state (e.g., after manually applying SQL, or when setting up Django migrations on a pre-existing database).

Mark all migrations as applied without running them:

```
python manage.py migrate --fake
```

Mark a specific migration as applied:

```
python manage.py migrate guards 0004_add_tier_field --fake
```

Fake-initial: mark initial migrations as applied if tables already exist.

```
# Safe to use when introducing Django migrations to an existing
# database.
python manage.py migrate --fake-initial
```

CI/CD Pre-Deploy Check

Run `migrate --check` before deploying. It exits with a non-zero code if there are unapplied migrations, preventing a deployment where the code expects a column that doesn't exist yet.

```
# In CI pipeline (pre-deploy step):
python manage.py migrate --check
# Exit code 0 → all migrations applied, safe to deploy
# Exit code 1 → unapplied migrations exist, block deployment

# .github/workflows/deploy.yml (excerpt)
- name: Check migrations are applied
  run: |
    python manage.py migrate --check
  env:
    DATABASE_URL: ${ secrets.STAGING_DATABASE_URL }
```

4. Zero-Downtime Migration Patterns

The Problem with ALTER TABLE on Large Tables

A naive

```
ALTER TABLE bookings_booking ADD COLUMN notes TEXT NOT NULL
DEFAULT ''
```

on a table with 10 million rows will:

1. Acquire an `AccessExclusiveLock` on the table (blocks all reads and writes)
2. Rewrite the entire table to add the column to every row
3. Hold the lock for minutes, causing a full application outage

Every migration in bSecure must be designed to avoid this.

Pattern 1: Adding a Nullable Column (Safe — No Lock)

Adding a nullable column with no default requires no table rewrite. PostgreSQL 11+ marks the column as null for existing rows in the catalog only.

```
# migrations/0010_add_notes_to_booking.py
from django.db import migrations, models
```

```

class Migration(migrations.Migration):
    dependencies = [("bookings", "0009_...")]

    operations = [
        # Safe: nullable, no default → zero table rewrite, near-
        # instant
        migrations.AddField(
            model_name="booking",
            name="notes",
            field=models.TextField(null=True, blank=True),
        ),
    ]

```

Pattern 2: Adding a NOT NULL Column with Default (Safe in PG 11+)

PostgreSQL 11 introduced an optimisation:

ADD COLUMN col TYPE NOT NULL DEFAULT const stores the default in the catalog and does NOT rewrite existing rows. Django's migration generates this exact SQL for simple constant defaults.

```

# migrations/0011_add_priority_to_booking.py
from django.db import migrations, models

```

```

class Migration(migrations.Migration):
    dependencies = [("bookings", "0010_...")]

    operations = [
        # Safe on PG 11+: constant default, NOT NULL → catalog-
        # only change
        migrations.AddField(
            model_name="booking",
            name="priority",
            field=models.IntegerField(default=0),
            # Django generates: ALTER TABLE bookings_booking
            # ADD COLUMN priority INTEGER NOT NULL DEFAULT 0;
            # PG 11+ stores default in pg_attrdef, no row rewrite.
        ),
    ]

```

Warning: This optimisation only applies to **constant defaults** (literals). A default that calls `now()`, `uuid_generate_v4()`, or any function is not constant and **will** cause a table rewrite on PG < 14. In that case, use the 3-step pattern below.

Pattern 3: NOT NULL Column on Older PG (3-Step)

For PostgreSQL < 11 or for non-constant defaults, use a 3-step migration spread across three separate deployments.

Step 1 — Add nullable column (deploy immediately):

```
# migrations/0012_add_cancellation_reason_nullable.py
operations = [
    migrations.AddField(
        model_name="booking",
        name="cancellation_reason",
        field=models.TextField(null=True, blank=True),
    ),
]
```

Step 2 — Backfill existing rows (data migration, deploy separately):

```
# migrations/0013_backfill_cancellation_reason.py
from django.db import migrations

def backfill_cancellation_reason(apps, schema_editor):
    Booking = apps.get_model("bookings", "Booking")
    Booking.objects.filter(
        status="cancelled",
        cancellation_reason__isnull=True
    ).update(cancellation_reason="No reason provided")

class Migration(migrations.Migration):
    dependencies = [("bookings", "0012_...")]

    operations = [
        migrations.RunPython(
            backfill_cancellation_reason,
            reverse_code=migrations.RunPython.noop,
        ),
    ]
```

Step 3 — Add NOT NULL constraint (deploy after backfill is verified):

```
# migrations/0014_set_cancellation_reason_not_null.py
from django.db import migrations, models

class Migration(migrations.Migration):
    dependencies = [("bookings", "0013_...")]

    operations = [
        migrations.AlterField(
            model_name="booking",
            name="cancellation_reason",
```



```

        field=models.TextField(default="No reason provided"),
    ),
]

```

Pattern 4: Renaming a Column (3-Step with SeparateDatabaseAndState)

Renaming a column is never safe in a single deployment: the old code references `old_name`, the new code references `new_name`. A direct rename causes downtime.

Use `SeparateDatabaseAndState` to decouple what Django's ORM thinks from what the database actually does.

Step 1 — Add new column, keep old column (dual-write at application level):

```

# migrations/0015_add_new_column_name.py
from django.db import migrations, models

class Migration(migrations.Migration):
    """
    Add the new column 'guard_notes' alongside the old 'notes'
    column.
    Application code must write to BOTH columns during this
    transition period.
    """
    dependencies = [("bookings", "0014_...")]

    operations = [
        migrations.AddField(
            model_name="booking",
            name="guard_notes",
            field=models.TextField(null=True, blank=True),
        ),
    ]

```

Step 2 — Copy data from old column to new column:

```

# migrations/0016_copy_notes_to_guard_notes.py
from django.db import migrations

def copy_notes(apps, schema_editor):
    Booking = apps.get_model("bookings", "Booking")
    Booking.objects.filter(
        notes__isnull=False,
        guard_notes__isnull=True
    ).update(guard_notes=models.F("notes")) # noqa: F821

```

```

class Migration(migrations.Migration):
    dependencies = [("bookings", "0015_...")]

    operations = [
        migrations.RunPython(copy_notes,
                              reverse_code=migrations.RunPython.noop),
    ]

```

Step 3 — Drop old column (after verifying all code references new column):

```

# migrations/0017_drop_old_notes_column.py
from django.db import migrations

```

```

class Migration(migrations.Migration):
    """
    Drop the old 'notes' column. Application code must no longer
    reference it.
    """
    dependencies = [("bookings", "0016_...")]

    operations = [
        migrations.RemoveField(model_name="booking",
                               name="notes"),
    ]

```

Using SeparateDatabaseAndState for an atomic ORM rename:

If you want Django's ORM to reference guard_notes while the DB still has notes (during the dual-write window), use SeparateDatabaseAndState:

```

# migrations/0015b_rename_notes_orm_only.py
from django.db import migrations

```

```

class Migration(migrations.Migration):
    """
    Tell Django's ORM that the field is now called 'guard_notes',
    but don't touch the database – the column is still named
    'notes'.
    """
    dependencies = [("bookings", "0014_...")]

    operations = [
        migrations.SeparateDatabaseAndState(
            # state_operations: what Django's migration state
            # (ORM) sees
            state_operations=[
                migrations.RenameField(
                    model_name="booking",
                    old_name="notes",

```

```

        new_name="guard_notes",
    ),
],
    # database_operations: what actually runs on the DB
    (nothing yet)
    database_operations=[],
),
]

```

Pattern 5: Adding an Index (CONCURRENTLY)

CREATE INDEX inside a Django migration runs inside a transaction by default. CREATE INDEX CONCURRENTLY **cannot** run inside a transaction. Set `atomic = False` on the migration.

```

# migrations/0018_add_index_booking_user_created.py
from django.db import migrations

```

```

class Migration(migrations.Migration):
    """
    Create an index on bookings_booking(user_id, created_at).
    Must use atomic=False because CONCURRENTLY cannot run in a
    transaction.
    """
    atomic = False # ← required for CONCURRENTLY

    dependencies = [("bookings", "0017_...")]

    operations = [
        migrations.RunSQL(
            sql="""
                CREATE INDEX CONCURRENTLY IF NOT EXISTS
                idx_bookings_booking_user_created
                ON bookings_booking (user_id, created_at DESC);
            """,
            reverse_sql="""
                DROP INDEX CONCURRENTLY IF EXISTS
                idx_bookings_booking_user_created;
            """,
        ),
    ]

```

Why `atomic = False`? By default, Django wraps each migration in a BEGIN/COMMIT. CONCURRENTLY requires the statement to run outside a transaction. Setting `atomic = False` disables the automatic wrapping. The trade-off: if the migration fails partway through, Django cannot roll back — you may be left with an invalid index (check with the query in `indexes.md`).

Pattern 6: Dropping a Column (3-Step)

Dropping a column immediately will break code that still references it (old workers still running, cached bytecode, etc.).

Step 1 — Ignore the column in Django (deploy first):

```
# In the model, mark the field as unused – do NOT remove it from  
migrations yet.  
# If using Django, you can temporarily add the field to a  
migration but  
# exclude it from model validation by adding it to Meta.managed =  
False  
# or by keeping it in the model but treating it as deprecated.
```

Step 2 — Remove field from Django model and generate migration:

```
# After the old code is fully retired from all nodes:  
python manage.py makemigrations guards # generates RemoveField  
migration
```

Step 3 — Apply the migration (the column is now dropped from the DB):

```
# migrations/0025_remove_guards_profile_old_photo_url.py  
from django.db import migrations
```

```
class Migration(migrations.Migration):  
    dependencies = [("guards", "0024_...")]  
  
    operations = [  
        migrations.RemoveField(model_name="guardprofile",  
                                name="old_photo_url"),  
    ]
```

Pattern 7: Changing a Column Type (New Column + Backfill + Swap)

Changing a column type (e.g., CharField → IntegerField) on a large table with a full table rewrite is dangerous. Use the new-column approach:

1. Add new column with the target type (nullable).
2. Backfill new column from the old column with a data migration.
3. Add NOT NULL constraint to the new column.
4. Rename columns (using SeparateDatabaseAndState pattern above).
5. Drop the old column.

This process spans 5 migrations across multiple deployments but guarantees zero-downtime.

5. Data Migrations

Rules for Data Migrations

1. **Always use `apps.get_model()`** — never import models directly. Direct imports reference the model's current definition, which may not match the schema at migration time.
2. **Batch large updates** — updating millions of rows in a single transaction takes an `AccessExclusiveLock` on the affected rows, bloats `pg_wal`, and can time out.
3. **Always write a reverse function** — even if it's noop. This documents the intent and allows `migrate app 000X` rollback in non-production environments.

Example 1: Backfilling `search_vector` `tsvector`

```
# migrations/0008_backfill_guard_search_vectors.py
from django.db import migrations

def backfill_search_vectors(apps, schema_editor):
    """
    Populate the search_vector tsvector column for all existing
    guard profiles.

    Processes in batches of 1000 to avoid long-running
    transactions.
    Uses .iterator() to avoid loading all objects into memory.
    """
    GuardProfile = apps.get_model("guards", "GuardProfile")
    db_alias = schema_editor.connection.alias

    # Fetch PKs in sorted order for deterministic batching
    pks = list(
        GuardProfile.objects.using(db_alias)
        .values_list("pk", flat=True)
        .order_by("pk")
    )

    batch_size = 1000
    for i in range(0, len(pks), batch_size):
        batch_pks = pks[i : i + batch_size]
        batch =
            GuardProfile.objects.using(db_alias).filter(pk__in=batch_pks)
```

```

for profile in batch.iterator():
    # Build tsvector manually (the trigger handles future
    # updates)
    schema_editor.execute(
        """
        UPDATE guards_guardprofile
        SET search_vector =
            setweight(to_tsvector('english', coalesce(%s,
            '')), 'A') ||
            setweight(to_tsvector('english', coalesce(%s,
            '')), 'B') ||
            setweight(to_tsvector('english', coalesce(%s,
            '')), 'C')
        WHERE id = %s
        """,
        [profile.full_name, profile.city, profile.skills,
        profile.pk],
    )

def reverse_backfill(apps, schema_editor):
    """Clear the search_vector column – safe to do on rollback."""
    GuardProfile = apps.get_model("guards", "GuardProfile")
    GuardProfile.objects.using(schema_editor.connection.alias).update(
        search_vector=None
    )

class Migration(migrations.Migration):
    dependencies = [("guards", "0007_add_search_vector_column")]

    operations = [
        migrations.RunPython(backfill_search_vectors,
            reverse_code=reverse_backfill),
    ]

```

Example 2: Migrating Booking Status Enum Values

```

# migrations/0012_rename_booking_status_values.py
from django.db import migrations

STATUS_MAPPING = {
    "in_progress": "active",
    "completed_by_guard": "completed",
    "cancelled_by_user": "cancelled",
}

REVERSE_STATUS_MAPPING = {v: k for k, v in STATUS_MAPPING.items()}

def migrate_status_values(apps, schema_editor):
    """

```

```

Rename booking status enum values to match the new
  standardised names.
  Batches updates by old status value to minimise lock
  contention.
"""
Booking = apps.get_model("bookings", "Booking")
db_alias = schema_editor.connection.alias

for old_status, new_status in STATUS_MAPPING.items():
    count = (
        Booking.objects.using(db_alias)
        .filter(status=old_status)
        .update(status=new_status)
    )
    print(f"  Renamed {count} bookings: {old_status!r} →
          {new_status!r}")

def reverse_status_values(apps, schema_editor):
    """Reverse the status renaming."""
    Booking = apps.get_model("bookings", "Booking")
    db_alias = schema_editor.connection.alias

    for new_status, old_status in REVERSE_STATUS_MAPPING.items():
        Booking.objects.using(db_alias).filter(status=new_status).update(
            status=old_status
        )

class Migration(migrations.Migration):
    dependencies = [("bookings", "0011_...")]

    operations = [
        migrations.RunPython(
            migrate_status_values,
            reverse_code=reverse_status_values,
        ),
    ]

```

Batching Large Data Migrations

```

def batch_update_large_table(apps, schema_editor):
    """
    Template for batching updates on tables with millions of rows.
    Uses keyset pagination (last_pk) instead of OFFSET for
    performance.
    OFFSET N on 10M rows is O(N) – keyset pagination is O(1).
    """

    MyModel = apps.get_model("myapp", "MyModel")
    db_alias = schema_editor.connection.alias

    last_pk = 0
    batch_size = 1000

```

```

total_updated = 0

while True:
    batch = list(
        MyModel.objects.using(db_alias)
        .filter(pk__gt=last_pk, some_field__isnull=True)
        .order_by("pk")[:batch_size]
    )

    if not batch:
        break

    for obj in batch:
        obj.some_field = compute_value(obj)

    MyModel.objects.using(db_alias).bulk_update(batch,
        ["some_field"])
    last_pk = batch[-1].pk
    total_updated += len(batch)

print(f" Updated {total_updated} rows")

```

6. PostGIS-Specific Migrations

Adding a Geography Column

```

# apps/guards/models.py
from django.contrib.gis.db import models as gis_models

class GuardProfile(models.Model):
    # ... other fields ...

    # geography=True stores coordinates in WGS84 and uses geodesic
    # (great-circle) distance calculations. srid=4326 is standard
    # GPS.
    current_location = gis_models.PointField(
        geography=True,
        srid=4326,
        null=True,
        blank=True,

        help_text="Guard's current GPS location (updated every 30s
        when online)",
    )

```


Generated Migration for a Geography Column

Running makemigrations after adding the PointField generates:

```
# migrations/0005_add_current_location.py
import django.contrib.gis.db.models.fields
from django.db import migrations

class Migration(migrations.Migration):
    dependencies = [("guards", "0004_add_tier_field")]

    operations = [
        migrations.AddField(
            model_name="guardprofile",
            name="current_location",
            field=django.contrib.gis.db.models.fields.PointField(
                blank=True,
                geography=True,
                null=True,
                srid=4326,
            ),
        ),
    ]
```

Django's PostGIS backend generates the correct SQL:

```
-- Generated SQL (PostGIS backend):
ALTER TABLE guards_guardprofile
    ADD COLUMN current_location geography(Point, 4326);
```

Creating the GiST Index (atomic=False Required)

```
# migrations/0006_add_gist_index_current_location.py
from django.db import migrations

class Migration(migrations.Migration):
    """
    Add GiST index on current_location for spatial queries.
    Must be atomic=False – CONCURRENTLY cannot run in a
    transaction.
    """
    atomic = False

    dependencies = [("guards", "0005_add_current_location")]

    operations = [
        migrations.RunSQL(
            sql="""
                CREATE INDEX CONCURRENTLY IF NOT EXISTS
                idx_guards_profile_location
            """
        )
    ]
```

```

        ON guards_guardprofile USING gist
        (current_location);
        """
        reverse_sql="""
        DROP INDEX CONCURRENTLY IF EXISTS
        idx_guards_profile_location;
        """
    ),
]

```

Complete Migration: Add current_location to guards_profile

```

# migrations/0005_add_current_location_full.py
"""

```

Full migration to add real-time location tracking to guards_profile.

Deployed as two separate migrations:

- 0005: add the column (nullable, no lock)
- 0006: add the GiST index (CONCURRENTLY, atomic=False)

This split ensures that if the index creation fails (e.g., disk full), the column already exists and the index can be recreated independently.

```

"""

```

```

import django.contrib.gis.db.models.fields
from django.db import migrations

```

```

class Migration(migrations.Migration):

```

```

    dependencies = [("guards", "0004_add_tier_field")]

```

```

    operations = [

```

```

        # Step 1: Add the geography column (safe, nullable, no
        # table rewrite)

```

```

        migrations.AddField(
            model_name="guardprofile",
            name="current_location",
            field=django.contrib.gis.db.models.fields.PointField(
                blank=True, geography=True, null=True, srid=4326
            ),
        ),

```

```

        # Step 2: Add is_online flag (guards broadcast location
        # only when online)

```

```

        migrations.AddField(
            model_name="guardprofile",
            name="is_online",

```

```

            field=django.contrib.gis.db.models.fields.BooleanField(default=False),
        ),

```

```

# Step 3: Add last_location_update timestamp for staleness
# detection
migrations.AddField(
    model_name="guardprofile",
    name="last_location_update",

    field=django.contrib.gis.db.models.fields.DateTimeField(
        null=True, blank=True
    ),
),
]

```

7. Squashing Migrations

When to Squash

Squash a Django app's migrations when:

- The app has more than **50 migrations**
- Running migrate from scratch (new environment, CI) takes more than **30 seconds** for that app
- The migration history has many back-and-forth changes (add field, remove field, add again) that can be collapsed

```

# Squash all migrations for the 'bookings' app into a single
# migration
# from 0001 through 0042:
python manage.py squashmigrations bookings 0042

# Squash from a specific starting point (e.g., after a previous
# squash):
python manage.py squashmigrations bookings 0030 0060

```

The replaces Attribute

Django generates a squashed migration with a replaces list that tells the framework which original migrations it supersedes:

```

# bookings/migrations/0001_squashed_0042.py
from django.db import migrations, models
import django.db.models.deletion

```

```

class Migration(migrations.Migration):

```

```

    # This squashed migration replaces all migrations from
    # 0001 to 0042.

```

```

# Once all production nodes have this squashed migration
# applied,
# the original 0001-0042 files can be deleted.
replaces = [
    ("bookings", "0001_initial"),
    ("bookings", "0002_add_payment_status"),
    # ... 40 more entries ...
    ("bookings", "0042_add_cancellation_timestamp"),
]

initial = True
dependencies = [
    ("users", "0001_squashed_0015"),
    ("guards", "0001_squashed_0022"),
]

operations = [
    # Collapsed set of operations that produce the final
    # schema
    migrations.CreateModel(
        name="Booking",
        fields=[
            # ... all fields as of 0042 ...
        ],
    ),
    # ... etc ...
]

```

Safe Deployment of Squashed Migrations

Step	Action	Notes
1	Run squashmigrations	Generates 0001_squashed_0042.py alongside original migrations
2	Test locally	migrate --run-syncdb on a fresh DB, verify schema matches
3	Deploy to staging	Django uses the squashed migration for new installs; existing installs use replaces to recognize it as applied
4	Deploy to production	Same — existing django_migrations rows for 0001-0042 satisfy the replaces list
5	Monitor for 1 week	Confirm no issues

Step	Action	Notes
6	Delete original migrations	Remove 0001 through 0042 files; remove replaces from squashed migration
7	Re-deploy	Now only the squashed migration exists

Never delete the original migrations before all nodes have the squashed migration applied. During a rolling deploy, some nodes may still be on the old migration state.

8. Migration Naming Conventions

Auto-Generated Names (Schema Migrations)

Auto-generated names like `0005_booking_payment_status` are acceptable for schema changes. Django derives them from the operations:

```
0001_initial
0002_guardprofile_add_tier
0003_booking_add_scheduled_start_time
0004_alter_booking_status
```

Descriptive Names (Data Migrations)

Data migrations must have descriptive names that communicate their purpose:

```
# Use --name to set a descriptive name:
python manage.py makemigrations guards --name
    backfill_guard_search_vectors --empty

# Results in:
# guards/migrations/0008_backfill_guard_search_vectors.py
```

Naming Conventions Table

Migration Type	Example Name	Convention
Initial table creation	<code>0001_initial</code>	Auto-generated
Add field	<code>0005_booking_add_notes</code>	

Migration Type	Example Name	Convention
Add index	0006_add_index_booking_user_created	Auto-generated Manual, descriptive
Data backfill	0007_backfill_guard_search_vectors	Manual, descriptive
Rename field	0008_rename_notes_to_guard_notes	Auto-generated
Status enum rename	0009_rename_booking_status_values	Manual, descriptive
Cross-app dependency	bookings_0010_fk_to_payments_wallet	Prefix with app name

9. Multi-Node Deployment

The Migration Lock Problem

When deploying to multiple nodes simultaneously, multiple processes may attempt to run `migrate` at the same time. Without coordination, this leads to:

- Race conditions on `django_migrations` table inserts
- Duplicate `CREATE TABLE / ALTER TABLE` errors
- Partial migration state on some nodes

Django's Built-In Migration Lock

Django's `MigrationExecutor` acquires a database-level advisory lock before applying migrations. As of Django 4.2, this is automatic:

```
# Django internals (for reference – you don't write this):
# MigrationExecutor.migrate() calls:
# connection.schema_editor().__enter__() which acquires:
# SELECT pg_advisory_lock(hash_record(('django', 'migrations')))
```

This means only one `manage.py migrate` process can apply migrations at a time. Other processes will wait, then detect that migrations are already applied and exit cleanly.

Health Check: Wait for Migrations Before Starting

In Kubernetes or Docker Compose, the application process should not start serving requests until migrations are complete. Use an `entrypoint.sh` script:

```
#!/bin/bash
# docker/entrypoint.sh

set -e

echo "==> Waiting for database to be ready..."
python manage.py wait_for_db # custom management command (see
                             below)

echo "==> Running database migrations..."
python manage.py migrate --noinput

echo "==> Collecting static files..."
python manage.py collectstatic --noinput --clear

echo "==> Starting Gunicorn..."
exec gunicorn bsecure.wsgi:application \
    --bind 0.0.0.0:8000 \
    --workers "${GUNICORN_WORKERS:-4}" \
    --worker-class gthread \
    --threads "${GUNICORN_THREADS:-2}" \
    --timeout 120 \
    --access-logfile - \
    --error-logfile -

# apps/core/management/commands/wait_for_db.py
import time
import django.db
from django.core.management.base import BaseCommand

class Command(BaseCommand):
    """Wait for the database to be available before proceeding."""

    help = "Waits for the database connection to be available"

    def handle(self, *args, **options):
        self.stdout.write("Waiting for database...")
        db_conn = None
        attempts = 0
        max_attempts = 30

        while not db_conn and attempts < max_attempts:
            try:
                django.db.connection.ensure_connection()
                db_conn = True
            except:
```

```

        except django.db.OperationalError:
            attempts += 1
            self.stdout.write(f" Database unavailable, retry
{attempts}/{max_attempts}...")
            time.sleep(2)

    if not db_conn:
        raise RuntimeError("Database connection failed after
30 attempts")

    self.stdout.write(self.style.SUCCESS("Database
available."))

```

Docker Compose Example

```

# docker-compose.yml (production-like)
services:
  web:
    build: .
    entrypoint: ["/app/docker/entrypoint.sh"]
    environment:
      - DATABASE_URL=postgis://bsecure:secret@db:5432/bsecure
    depends_on:
      db:
        condition: service_healthy

  worker:
    build: .
    # Worker must also wait for migrations – it may use models at
    # startup
    command: >
      sh -c "python manage.py migrate --check &&
        celery -A bsecure worker -l info"
    depends_on:
      web:
        condition: service_started

  db:
    image: postgis/postgis:15-3.3
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bsecure"]
      interval: 5s
      timeout: 5s
      retries: 10

```

10. Rollback Strategy

Django Has No Automatic Rollback

Unlike some migration tools, Django does not automatically revert schema changes on deploy failure. Rolling back requires explicitly running migrate to a previous migration:

```
# Roll back the last migration in the 'bookings' app:
python manage.py migrate bookings 0041

# Roll back ALL migrations for the 'bookings' app (drop all
  tables):
python manage.py migrate bookings zero
```

Warning: Never run `migrate bookings zero` in production. Use this only in development and for disaster recovery on a test database.

Nullable Columns for Easy Rollback

Every new column added to a production table should be nullable, even if it will eventually be NOT NULL. This makes rollback trivial: just remove the field from the model and generate a `RemoveField` migration — the column can be dropped without affecting existing data or requiring backfills.

```
# Prefer this (easy rollback):
field=models.TextField(null=True, blank=True)

# Over this (harder rollback – requires backfill before dropping):
field=models.TextField(default="")
```

Data Migration Rollback: Always Write a Reverse Function

```
def migrate_forward(apps, schema_editor):
    # ... make changes ...

def migrate_backward(apps, schema_editor):
    # ... undo the changes ...
    # Even if undoing is impossible (e.g., data was deleted),
    # document that with a comment rather than leaving
    # reverse_code=None.

operations = [
    migrations.RunPython(migrate_forward,
        reverse_code=migrate_backward),
]

# If rollback is genuinely impossible:
operations = [
```

```

migrations.RunPython(
    migrate_forward,
    reverse_code=migrations.RunPython.noop,
    # Add a comment explaining WHY rollback is a noop:
    # "Rollback not implemented: deleted rows cannot be
    # recovered.
    # To restore, deploy from the previous release tag."
),
]

```

Never Squash Without a Rollback Plan

Once a squashed migration is deployed to production and the original migrations are deleted:

- You **cannot** roll back to any of the original 0001–0042 migrations
- `migrate bookings 0020` will fail because 0020 no longer exists
- The only rollback path is restoring a database snapshot

Plan: Before deleting original migrations after squashing, verify that:

1. A database snapshot from before the squash is available and tested
 2. The next sprint does not include any schema changes that would require reverting
-

11. Migration Testing

Test Setup with pytest-django

```

# conf/test.py
import pytest

@pytest.fixture(scope="session")
def django_db_setup():
    """
    Use the default test database setup.
    pytest-django creates a test DB and runs all migrations
    automatically.
    """
    pass

# tests/test_models.py
import pytest
from apps.bookings.models import Booking

@pytest.mark.django_db

```

```
def test_booking_creation():
    """Basic sanity check that the migrations created a valid
       schema."""
    booking = Booking.objects.create(
        status="pending",
        # ... required fields ...
    )
    assert booking.pk is not None
    assert booking.status == "pending"
```

Test: All Migrations Are Applied (CI Health Check)

```
# tests/test_migrations.py
import pytest
from django.core.management import call_command
from io import StringIO

@pytest.mark.django_db
def test_no_pending_migrations():
    """
    Verify that all model changes have a corresponding migration.

    This test fails if a developer added a field to a model but
    forgot to
    run makemigrations. Catches schema drift before it reaches
    production.
    """
    out = StringIO()
    try:
        call_command(
            "migrate",
            "--check",
            stdout=out,
            stderr=StringIO(),
        )
    except SystemExit as e:
        # migrate --check exits with code 1 if there are unapplied
        # migrations
        pytest.fail(
            f"There are unapplied migrations. Run 'manage.py"
            f"migrate'.\n"
            f"Output: {out.getvalue()}"
        )
```

Test: No Missing Migrations (Detect Model Changes Without Migrations)

```
# tests/test_migrations.py (continued)
import pytest
from django.core.management import call_command
```

```

from io import StringIO

@pytest.mark.django_db
def test_no_missing_migrations():
    """
    Verify that all model changes have a corresponding migration
    file.

    This test fails if a developer added/changed a field in
    models.py
    but did not run makemigrations. Catches the "works on my
    machine"
    problem where a developer runs the app without migrations.
    """
    out = StringIO()
    err = StringIO()
    try:
        call_command(
            "makemigrations",
            "--check",
            "--dry-run",
            stdout=out,
            stderr=err,
        )
    except SystemExit as e:
        if e.code != 0:
            pytest.fail(
                f"Missing migrations detected. Run 'manage.py\n"
                f"makemigrations'.\n"
                f"Stdout: {out.getvalue()}\n"
                f"Stderr: {err.getvalue()}"
            )

```

Test: Migration Consistency for a Specific App

```

# tests/test_migrations.py (continued)
import pytest
from django.test.utils import override_settings
from django.db.migrations.executor import MigrationExecutor
from django.db import connection

@pytest.mark.django_db
def test_migration_plan_is_consistent():
    """
    Verify the migration dependency graph has no cycles or missing
    dependencies.
    Catches cross-app migration ordering issues.
    """
    executor = MigrationExecutor(connection)
    plan =
        executor.migration_plan(executor.loader.graph.leaf_nodes())

```

```

# If the plan is empty, all migrations are applied – that's
  fine in CI
# after migrate runs. This test is most useful when run
  against a fresh DB.
for migration, backwards in plan:
    assert not backwards, (
        f"Backwards migration in plan: {migration}. "
        "This indicates a dependency cycle."
    )

```

CI Pipeline Integration

.github/workflows/test.yml (excerpt)

```

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgis/postgis:15-3.3
        env:
          POSTGRES_DB: bsecure_test
          POSTGRES_USER: bsecure
          POSTGRES_PASSWORD: test_secret
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r requirements/test.txt

      - name: Check for missing migrations
        run: python manage.py makemigrations --check --dry-run
        env:
          DATABASE_URL: postgis://
            bsecure:test_secret@localhost:5432/bsecure_test

      - name: Run migrations
        run: python manage.py migrate --noinput
        env:
          DATABASE_URL: postgis://
            bsecure:test_secret@localhost:5432/bsecure_test

```

- name: Verify all migrations applied
run: python manage.py migrate --check
env:
 DATABASE_URL: postgis://
 bsecure:test_secret@localhost:5432/bsecure_test
- name: Run test suite
run: pytest --tb=short -q
env:
 DATABASE_URL: postgis://
 bsecure:test_secret@localhost:5432/bsecure_test